

Trabalho de Programação

Campeonato Computacional de Futebol – Parte II

Valor: 20 pontos
Deadline: 01 de julho de 2026

Prof. Thiago M. Paixão
thiago.paixao@ifes.edu.br

1 Objetivo

O trabalho prático de programação consiste em implementar um sistema simplificado de gerenciamento de dados de um campeonato computacional de futebol em linguagem C. Os dados são armazenados (persistidos) a partir de um arquivo CSV e carregados em uma estrutura de dados específica (memória) para execução do sistema. O campeonato é composto por 10 clubes (indexados de 0 a 9). São eles, respectivamente: JAVAlis, ESCorpiões, SemCTRL, GOrilas, PYthons, SeQueLas, NETunos, LOOPardos, RUSTicos, REACTivos.

A competição é disputada em um turno de pontos corridos, ou seja, todos jogam contra todos (sem repetição de jogos) e o campeão, obviamente, é o primeiro colocado. A classificação dos times é definida pelos seguintes critérios:

- **V**: vitórias;
- **E**: empates;
- **D**: derrotas;
- **GM**: gols marcados;
- **GS**: gols sofridos;
- **S**: saldo de gols ($GM - GS$);
- **PG**: pontos ganhos ($3V + E$);

Para esta entrega específica (Parte II), o foco se expande para a manutenção dos dados (inserção, remoção e atualização de partidas) e a ordenação da tabela de classificação. Será entregue um sistema completo baseado no TAD *LinkedList* (lista encadeada). As principais competências a serem desenvolvidas neste trabalho incluem:

- Uso e implementação listas encadeadas.
- Alocação dinâmica de memória.
- Modularização e TADs.
- Ordenação de dados.
- Manipulação de arquivos.
- Documentação da solução.

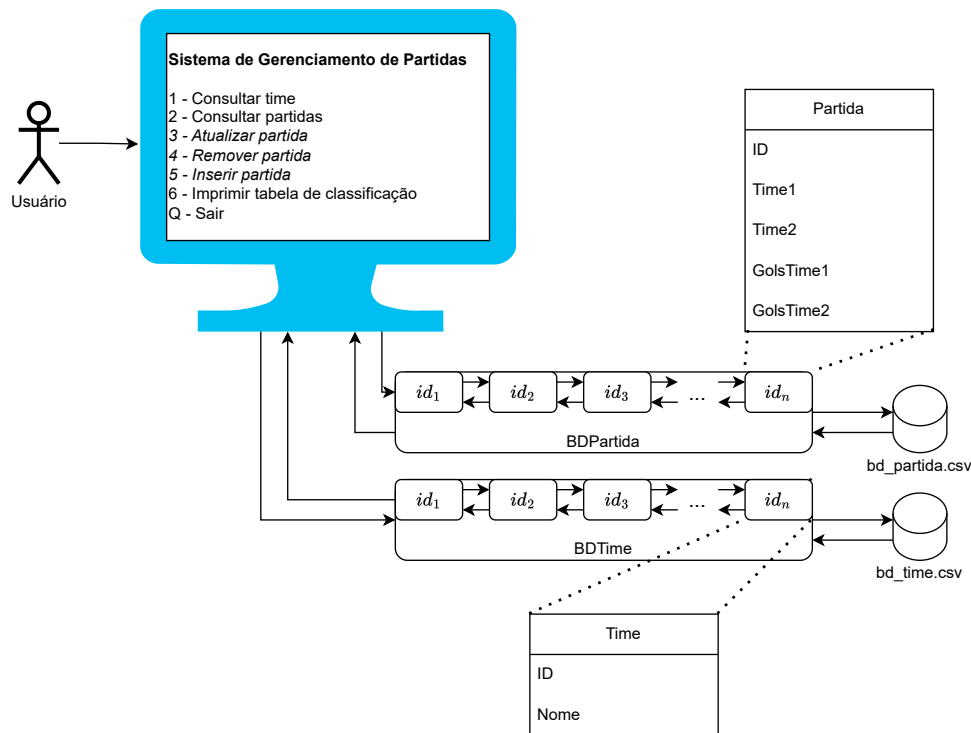


Figura 1: Sistema de gerenciamento de registro de partidas de futebol.

2 O Sistema

2.1 Comportamento básico

O sistema deve permitir o registro, atualização e remoção de partidas e resultados, além da consulta de dados e da tabela de classificação.

O comportamento básico do sistema é ilustrado na Figura 1. Ao rodar o sistema, o usuário acessa um menu com um total de seis opções, conforme indicado na figura. Diferente da etapa anterior, **todas as opções estarão habilitadas** nesta entrega. Deste modo, além das consultas (opções 1 e 2) e da impressão da tabela (opção 6), devem ser implementadas as funcionalidades de manutenção de dados: atualizar partida (3), remover partida (4) e inserir partida (5), detalhadas na seção 2.3.

Para este trabalho, simularemos um banco de dados composto por duas tabelas que persistem os dados em arquivos CSV. Ao iniciar o sistema, o conteúdo dos arquivos **times.csv** e **partidas.csv** será carregado para as estruturas de dados em memória responsáveis por gerenciar as informações de times e partidas, respectivamente. Essas estruturas funcionarão como um espelho dos arquivos CSV até que alguma operação de inserção, atualização ou exclusão seja realizada pelo usuário.

2.2 Banco de dados

2.2.1 Estrutura em arquivo

Para este trabalho, simularemos um banco de dados composto por duas tabelas que persistem os dados em arquivos CSV. Ao iniciar o sistema, os dados serão carregados dos arquivos **times.csv** e **partidas.csv**. O arquivo **times.csv** contém o identificador único (*id*) e o nome de cada time de futebol registrado. Já o arquivo **partidas.csv** armazena informações sobre os confrontos realizados, incluindo o nome do *time 1*, o *time 2*, o número de gols marcados por cada equipe e o resultado correspondente.

Arquivo times.csv

Este arquivo contém os dados referentes aos times de futebol cadastrados no sistema. Cada linha representa um time com os seguintes campos:

- ID: Identificador único de cada time garantindo que não existam duplicatas.
- Nome: Nome completo do time de futebol.

Arquivo **partidas.csv**

Este arquivo armazena os registros das partidas realizadas entre os times cadastrados. Cada linha representa uma partida com os seguintes campos:

- ID: Identificador único da partida.
- Time1: O ID do primeiro time participante.
- Time2: O ID do segundo time participante.
- GolsTime1: Quantidade de gols marcados pelo primeiro time.
- GolsTime2: Quantidade de gols marcados pelo segundo time.

Dessa forma, o banco de dados simula a relação entre as tabelas de times e partidas, permitindo que o sistema realize consultas e operações sobre as informações armazenadas de forma persistente.

```
ID,Time1,Time2,GolsTime1,GolsTime2
0,0,1,10,0
1,0,2,3,2
2,0,3,3,4
```

Figura 2: Exemplo de banco de dados em arquivo (CSV).

2.2.2 Conjuntos de Dados para Teste

Para garantir a robustez e a corretude do sistema, será disponibilizado três conjuntos de dados distintos para o arquivo de partidas. Cada conjunto representa um cenário específico da competição, permitindo validar o comportamento do programa em diferentes estágios. O arquivo **times.csv** permanece o mesmo em todos os cenários, enquanto o arquivo **partidas.csv** deve ser substituído por uma das seguintes versões para simular cada caso:

- **Cenário 1: Campeonato Vazio (**partidas_vazio.csv**)**
Este arquivo está vazio, contendo apenas o cabeçalho. Seu propósito é testar o comportamento inicial do sistema, onde nenhum jogo foi disputado. Espera-se que funcionalidades como "Imprimir tabela de classificação" exibam todos os times com suas estatísticas (pontos, vitórias, gols, etc.) devidamente zeradas.
- **Cenário 2: Campeonato em Andamento (**partidas_parcial.csv**)**
Contém um subconjunto das partidas totais da competição. Este cenário simula o campeonato durante seu andamento e serve como caso de teste para validar se os cálculos de pontos, saldo de gols e demais estatísticas estão sendo acumulados corretamente após cada partida processada.
- **Cenário 3: Campeonato Finalizado (**partidas_completo.csv**)**
Inclui o registro de todas as partidas planejadas. Este conjunto de dados permite verificar os resultados finais da competição e validar a totalização das estatísticas de todos os times. Também servirá como base para avaliar funcionalidades, como a ordenação da tabela de classificação final.

2.2.3 TAD Time

O TAD **Time** é uma abstração que modela uma única equipe de futebol, encapsulando tantos seus dados de identificação quanto seu desempenho na competição. Ele armazena os dados de identificação lidas do arquivo **times.csv** - um identificador único (ID) e um nome - e, de forma complementar, acumula as estatísticas de desempenho do time calculadas a partir dos resultados das partidas: vitórias, empates, derrotas, gols marcados (GM) e gols sofridos (GS).

A interface pública do TAD, além de fornecer acesso a esses dados, também oferece funções para obter dados derivados, como os pontos ganhos (PG) e o saldo de gols (SG), que são calculados sob demanda para garantir a consistência e a integridade das informações em todos os momentos. O TAD **Time** funciona como a estrutura de dados elementar que é criada, armazenada e manipulada pelo TAD BDTimes 2.2.4.

2.2.4 TAD BDTimes

O TAD **BDTimes** atua como um gerenciador para a coleção de todos os times (**Time** 2.2.3) do sistema. Diferente da etapa anterior, os dados agora são armazenados em uma **lista simplesmente encadeada**.

Sua principal responsabilidade é carregar os dados iniciais a partir do arquivo **times.csv**, criando uma instância do **TAD Time** para cada registro, e gerenciar a alocação dinâmica dos nós de times. Toda a manipulação dessa coleção - como busca, listagem ou atualização das estatísticas de um time específico - é realizado por meio de uma interface pública, que abstrai os detalhes de implementação.

2.2.5 TAD Partida

O TAD **Partida** é uma abstração para representar as informações de um único jogo de futebol. Ele encapsula todos os campos de um registro do arquivo **partidas.csv**: o identificador da partida (*ID*), os nomes dos times envolvidos (*Time 1* e *Time 2*), e a quantidade de gols marcada por cada um dos times (*GolsTime1* e *GolsTime2*). Este TAD serve como a estrutura de dados fundamental que será gerenciada pelo TAD BDPartidas 2.2.6.

2.2.6 TAD BDPartidas

O TAD **BDPartidas**, por sua vez, gerencia a **lista encadeada** de todas as partidas carregadas do arquivo **partidas.csv**. Sua interface deve oferecer um conjunto de operações para interagir com essa coleção, incluindo tanto as funções para busca e listagem de jogos já implementadas, quanto as funcionalidade de inserção, atualização e remoção de registros. A principal responsabilidade do TAD é fornecer ao sistema um acesso estruturado aos resultados dos jogos, permitindo que outras partes do sistema, como o **BDTimes** 2.2.4, processem esses dados para calcular as estatística de desempenho das equipes.

2.3 Funcionalidades

A seguir, são descritos os requisitos mínimos de funcionalidades do sistema. As descrições são fornecidas de forma geral, cabendo ao estudante decidir pela melhor forma de implementação considerando eficiência e usabilidade.

Consultar time A funcionalidade de consulta de time permite ao usuário verificar o desempenho detalhado de uma ou mais equipes na competição. A busca é realizada a partir do nome do time, suportando a pesquisa por prefixo. Para cada time encontrado que corresponda ao prefixo informado, o sistema exibirá um resumo completo de suas estatísticas, incluindo o número de vitórias (V), empates (E), derrotas (D), gols marcados (GM), gols sofridos (GS), o saldo de gols (S) e os pontos ganhos (PG).

Caso nenhum time seja encontrado com o nome ou prefixo fornecido, uma mensagem informativa de erro será exibida ao usuário. A figura 3 ilustra o fluxo de execução desta funcionalidade.

```
[Sistema]
Digite o nome ou prefixo do time:

[Usuário]
Se

[Sistema]
ID  Time          V   E   D   GM  GS  S   PG
2   SemCTRL      4   1   5   12  20 -8  13
5   SeQueLas     8   0   2   27  5  22  24
...
```

Figura 3: Simulação da execução de consulta de desempenho de time.

Consultar partidas A funcionalidade de consulta de partidas permite buscar os resultados das partidas utilizando o nome do time mandante, time visitante ou ambos. Baseado na informação fornecida pelo usuário, o sistema deve imprimir todos os registros correspondentes. Caso nenhuma partida seja encontrada, o sistema deve informar o usuário com uma mensagem de erro. A busca será baseada em prefixo, ou seja, se o usuário fornecer apenas uma parte do nome do time, o sistema deve retornar todos os registros que correspondam ao prefixo informado. A Figura 4 ilustra um possível fluxo de execução desta funcionalidade.

```
[Sistema]
Escolha o modo de consulta:
1 - Por time mandante
2 - Por time visitante
3 - Por time mandante ou visitante
4 - Retornar ao menu principal

[Usuário]
3

[Sistema]
Digite o nome:

[Usuário]
NET

[Sistema]
ID  Time1      Time2      Placar1      Placar2
23  NETunos    SemCTRL    1            1
47  REACTivos  NETunos    4            2
...
```

Figura 4: Simulação da execução de consulta de placares.

Atualizar partida A funcionalidade de atualização permite modificar os dados de uma partida existente no sistema. Para atualizar os dados de uma partida, é necessário inicialmente localizar um único registro de interesse. Dessa forma, a rotina de atualização deve primeiro invocar a rotina de consulta. Uma vez localizado, o registro pode ter alterado o placar entre um ou ambos os times envolvidos (**Placar1**, **Placar2**). Lembre-se, ao se editar, remover ou adicionar uma partida, as estatísticas dos times envolvidos podem impactar a tabela final diretamente, seja no saldo de gols, seja na pontuação final.

```
...

[Sistema]
ID  Time1      Time2      Placar1      Placar2
23  NETunos     SemCTRL    1            1
47  REACTivos   NETunos    4            2

Digite o ID do registro a ser atualizado:

[Usuário]
23

[Sistema]
Digite o novo valor para os campos Placar1 e Placar2 (para
manter o valor atual de um campo, digite '-'):

[Usuário]
7
-

[Sistema]
Confirma os novos valores para o registro abaixo? (S/N)

ID  Time1      Time2      Placar1      Placar2
23  NETunos     SemCTRL    7            1

[Usuário]
S

[Sistema]
Registro atualizado com sucesso.
```

Figura 5: Simulando a atualização de partida com confirmação.

Remover partida A funcionalidade de remoção permite excluir uma partida existente no sistema de forma definitiva. Para garantir que o registro correto seja excluído, o sistema deve solicitar inicialmente a execução de uma consulta para localizar a partida desejada. Após a seleção da partida, o sistema deve solicitar a confirmação do usuário antes de realizar a exclusão. A Figura 6 ilustra um possível fluxo de execução desta funcionalidade.

Inserir partida A funcionalidade de inserção permite adicionar uma nova partida ao sistema. Para cadastrar a partida, o sistema deve solicitar ao usuário que insira todos os campos obrigatórios, como **Time1**, **Time2**, **Placar1** e **Placar2**. O identificador da partida (**ID**) deve ser gerado automaticamente

```

...

[Sistema]
ID  Time1          Time2          Placar1          Placar2
23  NETunos        SemCTRL         1                1
47  REACTivos      NETunos         4                2

Digite o ID do registro a ser removido:

[Usuário]
23

[Sistema]
Tem certeza de que deseja excluir o registro abaixo? (S/N)

ID  Time1          Time2          Placar1          Placar2
23  NETunos        SemCTRL         1                1

[Usuário]
S

[Sistema]
Registro removido com sucesso.

```

Figura 6: Simulando a remoção de partida.

pelo sistema (ex: autoincremento) para garantir unicidade. Após o preenchimento, o sistema deve validar os dados fornecidos, como a existência do **Time1** e **Time2** e o preenchimento dos demais campos. Uma vez validados, os dados devem ser salvos no sistema, e o novo registro deve ser exibido ao usuário como confirmação. A Figura 7 ilustra um possível fluxo de execução desta funcionalidade.

Imprimir tabela de classificação A funcionalidade de impressão da tabela de classificação permite exibir todos os times cadastrados, mostrando o desempenho acumulado com base nas partidas realizadas. Nesta segunda parte, **é necessário ordenar a classificação** decrescentemente pelo mérito esportivo. A ordenação deve priorizar os Pontos Ganhos (PG) e utilizar critérios de desempate como Vitórias (V) e Saldo de Gols (S). Para cada time, o sistema deve apresentar os seguintes campos: ID, Time, V (vitórias), E (empates), D (derrotas), GM (gols marcados), GS (gols sofridos), S (saldo de gols) e PG (pontos ganhos). Cabendo ao aluno decidir qual método de ordenação será utilizado.

Caso haja um grande número de registros, é recomendado que a saída seja paginada para não sobrecarregar a interface. A Figura 8 ilustra a saída do sistema quando essa funcionalidade é solicitada.

3 Requisitos do programa

Neste trabalho, você terá a flexibilidade de implementar/adequar os módulos e os Tipos Abstratos de Dados (TADs) que considerar necessários para a simulação. No entanto, é fundamental que o programa principal esteja implementado no arquivo **main.c**. Além disso, a estrutura do código deve ser modular. É importante que cada módulo tenha uma responsabilidade clara e que a comunicação entre os diferentes componentes do programa ocorra de forma eficiente.

Note que, apesar das sugestões de implementação (fluxo de execução) em alto nível, os detalhes de

```
[Sistema]
Para inserir um novo registro, digite os valores para
os campos Time1, Time2, Placar1 e Placar2:
```

```
[Usuário]
```

```
0
1
0
6
```

```
[Sistema]
Confirma a inserção do registro abaixo? (S/N)
```

ID	Time1	Time2	Placar1	Placar2
90	JAVAlis	ESCorpiões	0	6

```
[Sistema]
0 registro foi inserido com sucesso.
```

Figura 7: Simulando a inserção de partida.

```
[Sistema]
Imprimindo classificação...
```

ID	Time	V	E	D	GM	GS	S	PG
0	JAVAlis	10	0	0	45	0	45	30
9	PYthons	9	0	1	52	4	48	27
2	SemCTRL	7	2	1	33	6	27	23
...								

Figura 8: Simulação da execução de impressão de lista de times e suas pontuações.

implementação devem ser decididos por vocês. Você é encorajado a adicionar e/ou modificar funcionalidades de modo a melhorar a experiência do usuário ou otimizar a execução. Não deve, contudo, reduzir a quantidade de funcionalidades já previstas, nem reduzir o escopo do projeto.

4 Critérios de avaliação

A avaliação deste trabalho levará em consideração os seguintes critérios que juntos somam **20 pontos**:

1. Funcionalidades: Até **15 pontos** serão atribuídos à implementação adequada das 5 funcionalidades requeridas (3 pontos por funcionalidade).
2. Lógica e organização do código: Até **2 pontos** serão concedidos pela clareza, organização e boas práticas de codificação no projeto. Isso inclui a estruturação adequada do código (modularização), nomes significativos para variáveis e funções, e formatação consistente.
3. Documentação do **README.md**: Até **2 pontos** serão atribuídos à qualidade do arquivo **README.md** presente no repositório. Este arquivo deve ser descritivo e informativo, contendo instruções claras sobre como executar e utilizar o projeto. Deve incluir informações detalhadas sobre a

estrutura do repositório, apresentar os principais TADs utilizados e listar as principais decisões de implementação tomadas ao longo do desenvolvimento.

4. Documentação interna do código: Até **1 ponto** será atribuído à qualidade da documentação incorporada diretamente ao código. Essa documentação deve ser composta por comentários significativos que expliquem a lógica por trás das implementações, facilitando a compreensão do funcionamento do projeto e promovendo a manutenção do código.
5. Robustez: A nota de robustez ($R \in [0, 1]$) será atribuída com base na presença de falhas críticas (por exemplo, falha de segmentação) ou não críticas (por exemplo, *memory leakage*) no sistema.
6. Dias de atraso: Serão contabilizados os dias de atraso (D) para efeito de desconto na nota total do trabalho.

A nota final do trabalho será calculada pela equação:

$$\text{nota} = \left(1 - \frac{2^D - 1}{31}\right) \times R \times P, \quad (1)$$

onde P é a soma dos pontos dos critérios 1 a 4. É importante ressaltar que a nota do trabalho será zerada caso o atraso ultrapasse 5 dias.

Importante: O programa será testado num ambiente Linux Ubuntu 22.04 com GCC 11. Recomendo FORTEMENTE desenvolver e testar nesse ambiente.

5 O que entregar?

1. Um link para um repositório .git com o arquivo **bd_partidas.csv**, **bd_classificacao.csv** e código-fonte do projeto: **Makefile** e arquivos **.c** e **.h** (Arquivos **.c** e **.h** podem estar guardados em pastas, sendo essencial que esteja informado no **Makefile** para a execução).
2. A documentação/relatório será feita no arquivo **README.md** do repositório e deverá explicar o passo-a-passo para executar o programa, os principais TADs e as principais decisões implementadas no código.

Bom trabalho!